𝕋v = v READING GROUP

Categorical types and AGI

Session B. The free category of a dynamic program and Bellman Algebra

Akshay Shanker

TBD

The question for today

Our goal is to **declare** the structure of a dynamic program using symbols, that is, names that can be written down in a **first-order** language, as in $x + y = z$ with $x : \mathbb{R}_+$.

- Yet recall from Session A that writing syntax for the higher-order Bellman operator is not straightforward, because its bound variables have no place in a first-order syntax tree.

We showed that the natural numbers give the initial dynamical system.

- Is there an initial category of the same kind for an arbitrary declared system, one that carries more of its structure?

Today's aim is to show recent work towards such a "**Bellman algebra**".

- To keep things simple for now, we restrict attention to sequential dynamic programs, without branching.

Notation: categories and graphs

- Categories are in sans-serif. $\mathsf{C}(x, y)$ is the set of arrows (morphisms) from x to y, and $g \circ f$ means f first, then g.

- Set is the category whose objects are sets and whose arrows are maps. Stoch is the category whose objects are measurable spaces and whose arrows are stochastic kernels, defined on the next slide.

- A **directed graph** has a set of vertices, a set of edges, and a source and a target vertex for each edge; parallel edges and loops are allowed. A **graph morphism** sends vertices to vertices and edges to edges, preserving sources and targets.

- Types and symbols are in typewriter and fields in italic. Sans-serif F and U are the free-category and underlying-graph constructions.

Riehl (2016) writes quiver for directed graph (p. 135). Italic *F* and *U* in Session C are maps between sets of value functions.

Notation: kernels and value functions

- A **stochastic kernel** $K : Y \rightsquigarrow Y'$ between measurable spaces assigns to each $y \in Y$ a probability measure $K(y, \cdot)$ on Y', with $y \mapsto K(y, B)$ measurable for every measurable B.
- Stoch has measurable spaces as objects and kernels as arrows, composed by $(K' \circ K)(y, B) = \int K(y, dy')\, K'(y', B)$, with identities $\delta_Y(y, B) = \mathbf{1}_B(y)$. For a measurable map f, $\delta_f(y, B) := \mathbf{1}_B(f(y))$ is the kernel that moves y to $f(y)$.
- $\Delta(X)$ is the set of probability measures on X. The **value line** is the extended real line $\bar{\mathbb{R}} = [-\infty, \infty]$, and L^X is the set of measurable functions $X \to \bar{\mathbb{R}}$, not required to be bounded.
- The **backward operator** of a kernel $K : Y \rightsquigarrow Y'$ sends $v \in L^{Y'}$ to $(K^* v)(y) := \int K(y, dy')\, v(y')$, defined where the integral exists. $\delta_Y^* = \mathrm{id}$ and $(K' \circ K)^* = K^* \circ K'^*$, so $K \mapsto K^*$ reverses the order of composition.

Fritz (2020), §4, pp. 18–19, equation (4.1) (Stoch on measurable spaces); Sargent and Stachurski (2026), §A.5.4.1, pp. 381–382 (kernels); §A.5.4.2, equation (A.26), p. 383 (the backward operator, there the Markov operator). An integral of a value function is taken where it exists, as when the function is bounded above or below; statements that use linearity are made for functions with finite integrals. The composition law follows from Tonelli's theorem applied to the positive and negative parts of *v*.

PART 1

1. Free categories and models

The initial discrete dynamical system · Signatures and fields · Interpretations · Forward and backward models

Discrete dynamical systems

Definition (discrete dynamical system). A **discrete dynamical system** is a triple (X, g, x_0), where

1. X is a set;
2. $g : X \to X$ is a map;
3. $x_0 \in X$ is a point.

Definition (morphism of systems). A **morphism** $(X, g, x_0) \to (Y, g', y_0)$ is a map $\varphi : X \to Y$ with

1. $\varphi \circ g = g' \circ \varphi$;
2. $\varphi(x_0) = y_0$.

With composition of maps, discrete dynamical systems and their morphisms form a category.

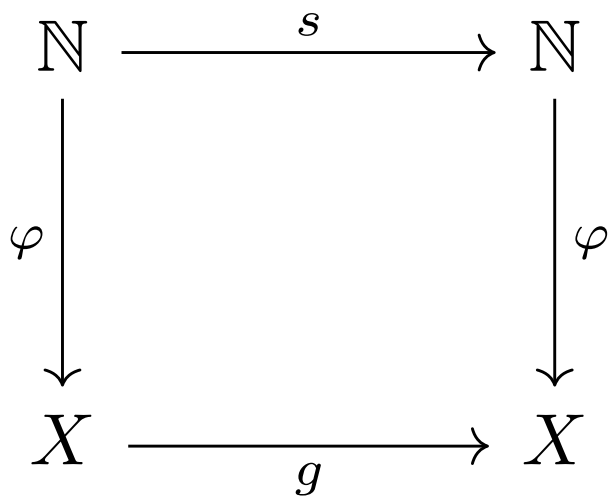
Riehl (2016), Example 2.1.1, p. 54; Example 2.4.11, p. 76.

The initial discrete dynamical system

- An object i of a category C is **initial** when for every object c there is exactly one arrow $i \to c$ (Riehl (2016), Definition 1.6.14).

- $(\mathbb{N}, s, 0)$, with $\mathbb{N} = \{0, 1, 2, \ldots\}$ and $s(n) = n + 1$, is initial. For every system (X, g, x_0) the one morphism is the trajectory $\varphi(n) = x_n$, where $x_{n+1} = g(x_n)$; recursion gives existence and induction on n uniqueness (Session A).

- The square asserts $\varphi \circ s = g \circ \varphi$.



Riehl (2016), Example 2.1.1 and diagram (2.1.2), pp. 54–55, whose diagram also contains a triangle asserting φ(0) = x_0; Definition 1.6.14, p. 44.

Example A: the buffer-stock problem

Consider a **standard** consumption–saving problem with parameters R (return factor), β (discount) and $\gamma > 1$ (CRRA), in which income $\xi_{t+1} > 0$ is drawn each period from a fixed law ν, independently of the past.

The agent's problem. With $u(c) = c^{1-\gamma}/(1-\gamma)$, the value functions satisfy

$$v_t(m_t) = \sup_{0<c_t\le m_t} u(c_t) + \beta\,\mathbb{E}\,v_{t+1}(m_{t+1}), \qquad m_{t+1} = R\,(m_t - c_t) + \xi_{t+1},$$

where m_t is cash, c_t is consumption and $a_t = m_t - c_t$ is savings; the expectation is over ξ_{t+1}.

The Bellman operator. In the stationary problem the value function solves $v = \mathbb{T}v$, where, for every v for which the expectation exists and every $m > 0$,

$$(\mathbb{T}v)(m) = \sup_{0<c\le m} \Big\{ u(c) + \beta\,\mathbb{E}_{\xi'}\, v\big(R(m-c) + \xi'\big) \Big\}.$$

Session A, "Buffer stock model with income growth", with a stochastic income and no growth in place of perfect foresight, and "The Bellman operator"; Carroll and Shanker (2026), Theoretical Foundations of Buffer Stock Saving, the problem ($\mathcal{P}_{\mathrm{L}}$) of Section 2. Examples are lettered as in fields-and-stages.md.

Example A: policies and the declaration

A **policy** is a measurable map σ with $0 \le \sigma(m) < m$. Out of cash m it saves $a = \sigma(m)$ and consumes the rest, $c = m - \sigma(m)$. Its **policy operator** is

$$(T_\sigma v)(m) = u(m - \sigma(m)) + \beta\, \mathbb{E}_{\xi'}\, v\big(R\,\sigma(m) + \xi'\big),$$

and $(\mathbb{T}v)(m) = \sup_\sigma (T_\sigma v)(m)$ for every m, because every savings level $a < m$ is the value at m of some policy, for instance $x \mapsto (a/m)\,x$.

One period of the problem has three quantities, cash m, savings a and next period's cash m', linked by two equations, the decision $a = \sigma(m)$ and the transition $m' = Ra + \xi'$. Later today we call these three quantities **fields**, and the quantities together with their equations a **declaration**.

A signature

Definition (type). A **type** is an element of a fixed set of symbols.

Definition (signature). A **signature** consists of

1. a set of types;
2. a set of names called **operator symbols**, distinct from the types;
3. for each symbol $\mathtt{s}$, a **rank**, which is a source type $\mathtt{X}$ and a target type $\mathtt{X}'$, written $\mathtt{s} : \mathtt{X} \to \mathtt{X}'$;
4. a subset of the operator symbols, called the **decision symbols**;
5. for each decision symbol $\mathtt{s}$, a nonempty set $\Sigma_{\mathtt{s}}$ whose elements are called **policies**.

Example A. Types $\mathtt{Xm}$ (cash) and $\mathtt{Xa}$ (savings); the decision symbol $\mathtt{save} : \mathtt{Xm} \to \mathtt{Xa}$, with $\Sigma_{\mathtt{save}}$ the policies of the previous slide, and the symbol $\mathtt{income} : \mathtt{Xa} \to \mathtt{Xm}$.

Goguen, Thatcher, Wagner and Wright (1977), §2, p. 70 (our types are their sorts, our rank their type ⟨w, s⟩ with w of length one); Shanker (2026), Appendix C of the Bellman calculus, §1.

Graphs of a signature

Definition (the graphs $\mathcal{S}$ and $\mathcal{S}_\Sigma$).

1. $\mathcal{S}$ is the directed graph whose vertices are the types and whose edges are the symbols, each $\mathtt{s} : \mathtt{X} \to \mathtt{X}'$ running from its source type to its target type.
2. $\mathcal{S}_\Sigma$ is the directed graph with the same vertices, one edge $(\mathtt{s}, \sigma) : \mathtt{X} \to \mathtt{X}'$ for each decision symbol $\mathtt{s} : \mathtt{X} \to \mathtt{X}'$ and each policy $\sigma \in \Sigma_{\mathtt{s}}$, and one edge $(\mathtt{s}, *) : \mathtt{X} \to \mathtt{X}'$ for each other symbol.

- **Example A.** $\mathcal{S}$ has two vertices and two edges; $\mathcal{S}_\Sigma$ has one edge $(\mathtt{save}, \sigma)$ for each policy, **parallel edges**, and the edge $(\mathtt{income}, *)$.

Terms

Definition (term). A **term** is a path in the graph $\mathcal{S}_\Sigma$:

1. a finite sequence of edges $\lambda_1, \ldots, \lambda_n$ in which the target type of each edge is the source type of the next, or the **empty path** at a type $\mathtt{X}$, a path with no edges that starts and ends at $\mathtt{X}$;
2. written $\lambda_n \cdots \lambda_1$, with the first edge on the right;
3. its **source type** is that of λ_1 and its **target type** that of λ_n.

Two terms are **parallel** when they have the same source type and the same target type.

- **Example A.** Writing $\mathtt{save}$ for any edge $(\mathtt{save}, \sigma)$, the terms from $\mathtt{Xm}$ to $\mathtt{Xm}$ are $(\mathtt{income\ save})^n$, $n \geq 0$. The term $\mathtt{income\ save}$ is one period of the buffer-stock problem, in which the household saves out of cash and then receives income.

The free category of the signature graph

Definition (free category $\mathsf{F}\mathcal{S}_\Sigma$). The **free category** $\mathsf{F}\mathcal{S}_\Sigma$ has

1. objects: the types;
2. arrows: the terms, a term from its source type to its target type;
3. composition: concatenation of paths;
4. identities: the empty paths.

- This is a category, because concatenation is associative and the empty path is an identity for it.

Riehl (2016), Example 4.1.13, p. 135 (the free category on a quiver).

Fields and declarations

Definition (declaration). A **declaration** over a signature consists of

1. a directed graph Q, whose vertices are called **fields**; field names are distinct from the types and the operator symbols;
2. a graph morphism $\tau : Q \to \mathcal{S}$, the **typing**, which assigns to each field j a type $\tau(j)$ and to each edge $e : j \to i$ a symbol $\tau(e)$ of rank $\tau(j) \to \tau(i)$.

Example A, one period. The fields are m, a and m'; the typing sends m and m' to $\mathtt{Xm}$, a to $\mathtt{Xa}$, the edge $m \to a$ to `save` and the edge $a \to m'$ to `income`.

Non-example. Typing the edge $m \to a$ by `income` is not a graph morphism, because `income` has rank $\mathtt{Xa} \to \mathtt{Xm}$ while $\tau(m) = \mathtt{Xm}$.

By analogy, a field resembles an atom of Backus's (1978) FP and FFP systems, a bare string (§11.2.1, p. 620): "both ρ and μ belong to the description of the system, not the system itself" (§13.3.1, p. 632).

Decision edges and the graph Q_Σ

Definition (decision edge, the graph Q_Σ).

1. An edge e of Q is a **decision edge** when $\tau(e)$ is a decision symbol, and **policy-free** otherwise.
2. Q_Σ is the directed graph with the fields as vertices, one edge $(e, \sigma) : j \to i$ for each decision edge $e : j \to i$ and each policy $\sigma \in \Sigma_{\tau(e)}$, and one edge $(e, *) : j \to i$ for each policy-free edge.
3. We write e_σ for (e, σ) and e for $(e, *)$.

Example A over N periods. For $0 \le t \le N-1$ the declaration has fields m_t (cash) of type `Xm` and a_t (savings) of type `Xa`, and decision edges $d_t : m_t \to a_t$ of symbol `save`; for $0 \le t \le N-2$ it has policy-free edges $\ell_t : a_t \to m_{t+1}$ of symbol `income`.

$$m_0 \xrightarrow{d_0} a_0 \xrightarrow{\ell_0} m_1 \xrightarrow{d_1} a_1 \xrightarrow{\ell_1} \cdots \longrightarrow a_{N-1}$$

Example E: the consumption–portfolio problem

In each period the agent holds wealth $k \geq 0$ and, before the returns are drawn, allocates a share $\sigma_1(k) \in (0,1)$ of it to a risky asset and the rest to a safe asset with gross return $R > 0$. Then the risky gross return $\Psi > 0$ and the income $\theta > 0$ are drawn from a fixed joint law, independently across periods, and cash is $m = k[\sigma_1(k)\Psi + (1-\sigma_1(k))R] + \theta$.

The agent then chooses consumption $c \in (0, m]$, receives the reward $u(c)$, and carries wealth $k' = m - c$ into the next period; a reward one period ahead is discounted by $\beta \in (0,1)$.

A policy of the portfolio decision is a measurable share policy σ_1 with values in $(0,1)$; a policy of the consumption decision is a measurable consumption policy σ_2 with $0 < \sigma_2(m) \leq m$. We write $\sigma = (\sigma_1, \sigma_2)$ for a pair. Unlike the policy of Example A, which chooses savings, σ_2 chooses consumption; next wealth is $m - \sigma_2(m)$.

Example E: signature and declaration

Types $\mathtt{Xk}$ (wealth) and $\mathtt{Xm}$ (cash); two decision symbols, $\mathtt{portfolio} : \mathtt{Xk} \to \mathtt{Xm}$ for the cash equation, with the share policies as its policies, and $\mathtt{consume} : \mathtt{Xm} \to \mathtt{Xk}$ for $k' = m - c$, with the consumption policies. For each date t the declaration has fields k_t and m_t and decision edges $k_t \to m_t$ and $m_t \to k_{t+1}$.

$$k_t \xrightarrow{\mathtt{portfolio}_{\sigma_1}} m_t \xrightarrow{\mathtt{consume}_{\sigma_2}} k_{t+1}$$

Writing the symbols without their policies, the terms from $\mathtt{Xk}$ to $\mathtt{Xk}$ are $(\mathtt{consume}\ \mathtt{portfolio})^n$ and the terms from $\mathtt{Xm}$ to $\mathtt{Xm}$ are $(\mathtt{portfolio}\ \mathtt{consume})^n$. A period can start at either field.

The free category of a graph

Definition (free category of a graph). The **free category** $\mathsf{F}Q$ of a directed graph Q has

1. objects: the vertices of Q;
2. arrows: the finite paths of Q;
3. composition: concatenation;
4. identities: the empty paths.

Definition (underlying graph, the inclusion η).

1. UC is the graph of objects and arrows of a category C; U sends a functor to its action on objects and arrows.
2. $\eta : Q \to \mathsf{UF}Q$ is the graph morphism sending each vertex to itself and each edge to its path of length one.

Riehl (2016), Example 4.1.13, p. 135. When C is large, the vertices and edges of UC form classes rather than sets.

Interpretations

Definition (interpretation). An **interpretation** of a directed graph Q is a pair (C, χ), where

1. C is a category;
2. χ assigns an object $\chi(j)$ of C to each vertex j and an arrow $\chi(e) : \chi(j) \to \chi(i)$ to each edge $e : j \to i$.

Definition (morphism of interpretations). A **morphism of interpretations** $(\mathsf{C}, \chi) \to (\mathsf{C}', \chi')$ is a functor $H : \mathsf{C} \to \mathsf{C}'$ with

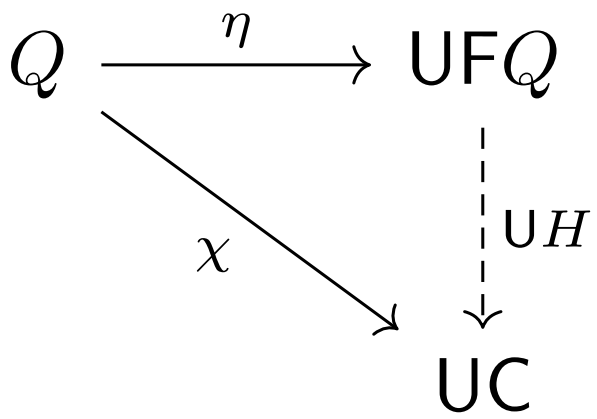
1. $H(\chi(j)) = \chi'(j)$ for every vertex j;
2. $H(\chi(e)) = \chi'(e)$ for every edge e.

An interpretation assigns an arrow to each edge. No equation between the arrows of different edges is required, because a graph has no composites to preserve.

Riehl (2016), Example 4.1.13, p. 135 (an interpretation is a diagram "with no commutativity requirements"); Definition 1.6.4, p. 40.

Proposition 1

Proposition 1. For every category C and every interpretation (C, χ) of Q there is exactly one functor $H : \mathsf{F}Q \to \mathsf{C}$ with $\mathsf{U}H \circ \eta = \chi$, that is, with $H(j) = \chi(j)$ and $H(e) = \chi(e)$ for every vertex and edge. In this sense $(\mathsf{F}Q, \eta)$ is initial among the interpretations of Q.



The triangle is a diagram of graphs and graph morphisms; it asserts $\mathsf{U}H \circ \eta = \chi$, and the dashed arrow is the functor that exists and is unique.

Riehl (2016), Example 4.1.13, p. 135 (functors out of the free category correspond to quiver morphisms). When the arrows of C form a set, the proposition is Riehl's Theorem 4.2.7(v), pp. 138–139, which states that η is initial in the comma category Q ↓ U; the proof on the next slide needs no such restriction, so Set and Stoch are admitted.

Proof of Proposition 1: uniqueness

Let H agree with χ on vertices and edges. A path of positive length is the composite of its edges, so preservation of composites gives $H(e_n \cdots e_1) = \chi(e_n) \circ \cdots \circ \chi(e_1)$; the empty path at j is the identity of j, so preservation of identities gives $H(\text{empty path at } j) = \mathrm{id}_{\chi(j)}$. Hence H is determined by χ.

Proof of Proposition 1: existence

1. Define H by the two formulas of the previous slide. The composite $\chi(e_n) \circ \cdots \circ \chi(e_1)$ is defined because χ preserves sources and targets.

2. H preserves identities by the second formula, and composites because concatenation goes to composition, by associativity when both paths are nonempty and by the identity law when one is empty.

3. On a path of length one, $H(e) = \chi(e)$, so $\mathsf{U}H \circ \eta = \chi$. ■

Algebras

Definition (algebra). An **algebra** of the signature in a category C assigns

1. to each type $\mathtt{X}$ an object $|\mathtt{X}|$ of C, its **carrier**;
2. to each edge $\lambda : \mathtt{X} \to \mathtt{X}'$ of $\mathcal{S}_\Sigma$ an arrow $|\lambda| : |\mathtt{X}| \to |\mathtt{X}'|$ of C.

By Proposition 1 the assignment extends to exactly one functor $\mathsf{F}\mathcal{S}_\Sigma \to \mathsf{C}$, denoted by the same letter.

- **Example A in Stoch.** The carriers $|\mathtt{Xm}|$ and $|\mathtt{Xa}|$ are measurable spaces, and $|(\mathtt{save}, \sigma)|$ and $|\mathtt{income}|$ are stochastic kernels between them.

An algebra is a many-sorted algebra with one input per symbol in the sense of Goguen, Thatcher, Wagner and Wright (1977), §2, p. 70, whose word for type is sort.

Models

Definition (model). A **model** of a declaration Q in C assigns

1. to each field j an object $D(j)$ of C, the **carrier** of the field;
2. to each edge $(e, \sigma) : j \to i$ of Q_Σ an arrow $D(e, \sigma) : D(j) \to D(i)$ of C.

By Proposition 1 the assignment extends to exactly one functor $\mathsf{F}Q_\Sigma \to \mathsf{C}$, denoted by the same letter.

- An algebra **induces** a model, in which the field j gets the carrier $|\tau(j)|$ of its type and the edge (e, σ) gets the arrow $|(\tau(e), \sigma)|$ of its symbol with the same policy.

Propositions are numbered as in the note fields-and-stages.md; its Proposition 2 is not used.

Models as denotational semantics

1. A program is processed in four steps:
    i. lexing splits the text into tokens;
    ii. parsing arranges the tokens into a syntax tree;
    iii. **semantics** assigns a meaning to the tree;
    iv. compilation translates the program into executable code.

A semantics is **denotational** when it assigns to each expression a mathematical object, its **denotation**, rather than describing **how** a machine executes the expression.

- The denotation of a field is its carrier, and the denotation of an edge is the arrow that the model assigns to it.
- The forward model assigns push-forwards of probability laws; the backward model assigns the operators of value iteration.

A further elaboration can assign numerical detail.

The forward model

A **forward model** maps:

- each field to a measurable space, its **state space**,
- and each edge to a stochastic kernel between the state spaces of its two fields, that is, to a real-valued function $K(y, B)$ giving the probability of landing in the set B when starting from the point y.

Example A, one period. The fields are m, a and m'; the decision edge $d : m \to a$ is the equation $a = \sigma(m)$, and the income edge $\ell : a \to m'$ is the equation $m' = Ra + \xi'$.

	IN GENERAL	EXAMPLE A
field j	its state space X_j	$m, m' \mapsto M = (0, \infty);\ a \mapsto A = [0, \infty)$
edge $(e, \sigma) : j \to i$	a kernel $X_j \rightsquigarrow X_i$	$d_\sigma \mapsto \delta_\sigma;\ \ell \mapsto P, P(a, B) := \nu\{\xi : Ra + \xi \in B\}$

Formally, a forward model is a functor $D_f : FQ_\Sigma \to \mathrm{Stoch}$. A kernel K pushes a law μ on Y forward to the law μK on Y', (μK)(B) = ∫ μ(dy) K(y, B).

The forward model of Example E

1. **State spaces.** Every wealth field k_t goes to $X_k = [0, \infty)$ and every cash field m_t to $X_m = (0, \infty)$, with their Borel sets.

2. **The portfolio edge** $(\texttt{portfolio}, \sigma_1) : k_t \to m_t$ goes to the kernel P_{σ_1} with $P_{\sigma_1}(k, \cdot)$ the law of $k[\sigma_1(k)\Psi + (1 - \sigma_1(k))R] + \theta$; the share is chosen first, then the returns and income are drawn.

3. **The consumption edge** $(\texttt{consume}, \sigma_2) : m_t \to k_{t+1}$ goes to the point-mass kernel δ_{σ_2} of the map $m \mapsto m - \sigma_2(m)$; next wealth is what is not consumed.

4. **A period** $k_t \to m_t \to k_{t+1}$ goes to $\delta_{\sigma_2} \circ P_{\sigma_1}$, which sends wealth k to the law of $m - \sigma_2(m)$ with m drawn from $P_{\sigma_1}(k, \cdot)$, since $(\delta_{\sigma_2} \circ P_{\sigma_1})(k, B) = P_{\sigma_1}(k, \{m : m - \sigma_2(m) \in B\})$. Both edges are decision edges, so a pair $\sigma = (\sigma_1, \sigma_2)$ fixes one path per period.

The backward model

A **backward model** maps:

- each field j to the set L^{X_j} of value functions on its state space,
- and each edge $(e, \sigma) : j \to i$ to an operator carrying a value function on the target field i to a value function on the source field j
  - in the time-separable case this operator is $v \mapsto r_{e,\sigma} + \beta_e \, K^* v$, for a **reward** $r_{e,\sigma} \in L^{X_j}$, zero on policy-free edges, a **discount factor** $\beta_e > 0$, and the kernel K of the edge under the forward model.

	IN GENERAL, FOR TIME-SEPARABLE PROBLEMS	EXAMPLE A
field j	L^{X_j}, the value functions on X_j	$m, m' \mapsto L^M;\ a \mapsto L^A$
edge $(e, \sigma) : j \to i$	$v \mapsto r_{e,\sigma} + \beta_e \, K^* v$, from L^{X_i} to L^{X_j}	$\ell \mapsto P^*;\ d_\sigma \mapsto (w \mapsto u(m - \sigma(m)) + \beta \, w(\sigma(m)))$
path	the composite, in reverse order	the period $\mapsto D_b(d_\sigma) \circ D_b(\ell) = T_\sigma$

Formally, a backward model is a functor $D_b : (FQ_\Sigma)^{op} \to \mathbf{Set}$; the opposite category appears because the operators run against the edges. Riehl (2016), Definition 1.2.1, p. 9 (the opposite category); Definition 1.3.5, p. 18 (a functor out of it is a contravariant functor).

The backward model of Example E

1. **Rewards and discounting.** The consumption edge carries the reward $r(m) = u(\sigma_2(m))$ and the discount factor β; the portfolio edge carries reward 0 and discount factor 1.

2. **The consumption edge** goes to $\mathbf{M}_\sigma : L^{X_k} \to L^{X_m}$, $(\mathbf{M}_\sigma v)(m) = u(\sigma_2(m)) + \beta\, v(m - \sigma_2(m))$, which is the reward plus the discounted value of next wealth.

3. **The portfolio edge** goes to $\mathbf{P}_\sigma : L^{X_m} \to L^{X_k}$, $(\mathbf{P}_\sigma v)(k) = \mathbb{E}\, v\big(k[\sigma_1(k)\Psi + (1 - \sigma_1(k))R] + \theta\big) = (P^*_{\sigma_1} v)(k)$, which is the expectation of the value of cash over the returns and income.

4. **A period** $k_t \to k_{t+1}$ goes to $\mathbf{P}_\sigma \circ \mathbf{M}_\sigma$ on L^{X_k}, in the reverse order of the edges, so that $(\mathbf{P}_\sigma \mathbf{M}_\sigma w)(k) = \mathbb{E}\big[u(\sigma_2(m)) + \beta\, w(m - \sigma_2(m))\big]$ with $m = k[\sigma_1(k)\Psi + (1 - \sigma_1(k))R] + \theta$; this is the policy operator of the problem on functions of wealth.

Both models are functors out of the free category

$$m \xrightarrow{d_\sigma} a \xrightarrow{\ell} m'$$

$$M \overset{\delta_\sigma}{\rightsquigarrow} A \overset{P}{\rightsquigarrow} M$$

$$L^M \xleftarrow{D_b(d_\sigma)} L^A \xleftarrow{D_b(\ell)=P^*} L^M$$

- **Top row:** the graph Q_Σ of the one-period declaration, for one policy σ; its free category is the domain of both models.
- **Middle row:** the forward model, along the edges, sending fields to state spaces and edges to kernels.
- **Bottom row:** the backward model, against the edges, sending fields to sets of value functions and edges to operators.

No vertical arrow joins the rows, so the diagram asserts no equation.

Natural transformations

Definition (natural transformation). Let $D, D' : \mathsf{C} \to \mathsf{E}$ be functors. A **natural transformation** $\alpha : D \Rightarrow D'$ consists of

1. for each object c of C, an arrow $\alpha_c : D(c) \to D'(c)$ of E, its **component** at c;
2. such that $D'(f) \circ \alpha_c = \alpha_{c'} \circ D(f)$ for every arrow $f : c \to c'$ of C; this equation is the **naturality square** at f.

- A natural transformation relates two functors with the same domain and the same codomain. $D_f : \mathsf{F}Q_\Sigma \to \mathsf{Stoch}$ and $D_b : (\mathsf{F}Q_\Sigma)^{\mathrm{op}} \to \mathsf{Set}$ differ in both, so no natural transformation between them is defined.

- **Example from the stock.** For vector spaces, the maps $V \to V^{**}$ sending a vector to evaluation at it are the components of a natural transformation from the identity functor to the double-dual functor (Riehl (2016), Example 1.4.4(i), p. 25).

Riehl (2016), Definition 1.4.1, p. 25.

Pairing of laws and value functions

1. Write $\langle \mu, v \rangle := \int v \, d\mu$ for $\mu \in \Delta(X)$ and $v \in L^X$ with a finite integral.

2. For every kernel $K : X \rightsquigarrow Y$, every $\mu \in \Delta(X)$ and every $v \in L^Y$ with $\int \mu(dx) \int K(x, dy) \, |v(y)| < \infty$,

$$\langle \mu K, v \rangle = \langle \mu, K^* v \rangle,$$

   since both sides equal $\int \mu(dx) \int K(x, dy) \, v(y)$ by Fubini's theorem.

3. For finite state spaces K is a stochastic matrix, μK is the row vector μ times K, $K^* v$ is K times the column vector v, and the identity reads $(\mu K) v = \mu (K v)$.

Sargent and Stachurski (2026), Lemma A.5.33, equation (A.30), p. 385, for a kernel on one space.

The pairing is natural

1. Sending μ to the functional $v \mapsto \langle \mu, v \rangle$ is a natural transformation from the functor $j \mapsto \Delta(X_j)$, on which a path p acts by $\mu \mapsto \mu D_f(p)$, to the functor $j \mapsto \mathsf{Set}(L^{X_j}, \mathbb{R})$, on which p acts by $\phi \mapsto \phi \circ D_f(p)^*$. The second functor is covariant because it reverses arrows twice, and the naturality square at a path with kernel K is the identity $\langle \mu K, v \rangle = \langle \mu, K^* v \rangle$.

2. Equivalently, the pairings form a **dinatural transformation**: a family of maps $\langle \cdot, \cdot \rangle_j$ on $L^{X_j} \times \Delta(X_j)$, one for each field, such that $\langle K^* v, \mu \rangle_j = \langle v, \mu K \rangle_i$ for every path $j \to i$ with kernel K; this is the form naturality takes when one argument runs against the arrows and the other along them. The condition holds at every path because the identity holds for every kernel.

Item 1 is the analogue of evaluation into the double dual of a vector space, Riehl (2016), Example 1.4.4(i), p. 25; Exercise 1.4.viii, p. 30, and footnote 32, p. 26 (dinatural transformations).

Next session

1. **Stages.** Grouping the edges of a model into stages, each with one decision edge.
    i. Natural transformations of **stagings**, which generalize Sargent and Stachurski's (2026) factored programs as a naturality square.
2. **Branching.** Formalizing an algebra for models with paths that lead to independent sub-problems.
3. **The Yoneda lemma**, and what it says about policies and about operations on value functions.

References

- Backus, John (1978). "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs." *Communications of the ACM* 21(8), 613–641.
- Fritz, Tobias (2020). "A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics." *Advances in Mathematics* 370, 107239; arXiv:1908.07021, pages of the arXiv version.
- Goguen, J. A., J. W. Thatcher, E. G. Wagner, and J. B. Wright (1977). "Initial Algebra Semantics and Continuous Algebras." *Journal of the ACM* 24(1), 68–95.

References

- Riehl, Emily (2016). *Category Theory in Context*. Dover.
- Sargent, Thomas J., and John Stachurski (2026). *Dynamic Programming, Volume II: General States*, draft of 10 April 2026.
- Shanker, A. (2026). *Factoring dynamic programs: semi-conjugate sliders*, talk, `bellman-ddsl/AI/dev-docs/presentation/abstract-programs/abstract-programs-talk.md`. The Bellman calculus, Appendix C (`bellman-ddsl/docs/bellman-calculus/APPX-C-STAGE-TERM.md`).